

Intégration Continue (CI/CD) et Automatisation des Tests

Ce document marque l'aboutissement de l'industrialisation des scripts PowerShell. Il aborde la transition entre l'exécution manuelle (sur le poste de l'administrateur) et l'exécution automatisée (*headless*) au sein d'un pipeline d'Intégration et de Déploiement Continus (CI/CD). Il détaille la gestion des codes de retour, l'exportation des résultats Pester pour les orchestrateurs, et la structuration d'un flux de travail DevOps.

1. Le Changement de Paradigme : Qu'est-ce que la CI/CD pour l'Infra ?

Historiquement, un administrateur écrit un script, le teste sur son poste, puis l'exécute en production depuis sa propre session (ou via une tâche planifiée locale).

Dans une approche **DevOps / GitOps**, l'administrateur n'exécute plus les scripts lui-même :

1. Il écrit le code PowerShell (Script, Module, ou Tests Pester).
2. Il "pousse" (Push) ce code sur un dépôt Git centralisé (GitLab, GitHub, Azure DevOps).
3. Cet envoi déclenche automatiquement un **Pipeline CI/CD**.
4. Un serveur esclave jetable (appelé *Runner* ou *Agent*) télécharge le code, exécute les tests, et si tout est vert, se charge de l'exécuter en production.

Conséquence majeure : Le script PowerShell doit être capable de s'exécuter de manière 100% autonome, sans aucune interaction humaine, et de communiquer son succès ou son échec à la machine (le Runner) qui l'exécute.

2. Le Langage Universel de la CI/CD : Les Codes de Retour (Exit Codes)

Un Runner CI/CD (qu'il soit sous Linux ou Windows) ne sait pas lire. Si votre script PowerShell affiche une erreur rouge via Write-Error, mais termine son exécution, le Runner considérera que le script a réussi (Statut : **Success**) et déploiera potentiellement une infrastructure cassée.

La pipeline se fie uniquement au **Code de Sortie (Exit Code)** du processus powershell.exe ou pwsh.

- 0 : Succès total.
- 1 (ou tout autre nombre différent de 0) : Échec critique.

2.1 Maîtriser le code de sortie d'un script

Pour qu'un script d'infrastructure communique correctement avec la pipeline, il doit encapsuler sa logique dans un bloc try/catch avec un arrêt strict sur erreur (-ErrorAction Stop) et forcer

l'utilisation du mot-clé exit.

Script de déploiement (deploy.ps1) exécuté par la CI/CD

```
try {
    Write-Host "Déploiement du service en cours..."
    # L'utilisation de -ErrorAction Stop est vitale ici
    Start-Service -Name "ServiceMetier" -ErrorAction Stop

    Write-Host "Déploiement réussi." -ForegroundColor Green
    # On force le code 0 pour valider le Job (Success)
    exit 0
}
catch {
    Write-Host "ÉCHEC CRITIQUE : $($_.Exception.Message)" -ForegroundColor Red

    # On force un code d'erreur pour faire échouer le Job (Failed)
    exit 1
}
```

3. Paramétrage Dynamique : L'Usage des Variables d'Environnement

Dans une CI/CD, on ne demande pas à un humain de taper un mot de passe ou de choisir un serveur cible. Ces données sont configurées dans l'interface de l'orchestrateur (GitLab CI / GitHub Settings) et sont injectées dans la mémoire du Runner au démarrage.

Le script PowerShell doit donc récupérer ces données via le fournisseur Env: :

```
[CmdletBinding()]
param(
    # Le script va chercher la variable $env:DEPLOY_TARGET injectée par la pipeline
    # S'il ne la trouve pas, il utilise "localhost" par défaut
    [string]$ServeurCible = $env:DEPLOY_TARGET,

    # Même principe pour les identifiants ou numéros de build
    [string]$BuildVersion = $env:CI_PIPELINE_ID
)

Write-Host "Déploiement de la version $BuildVersion sur l'hôte $ServeurCible..."
```

(Note : Pour les mots de passe et API Keys, on couplera cette technique avec le module SecretStore vu au J4S3 pour ne jamais exposer les valeurs en clair dans les logs).

4. Intégration de Pester dans la CI/CD (L'Usine de Test)

L'objectif principal d'une pipeline CI/CD pour l'infrastructure est d'exécuter vos tests Pester de manière automatisée à chaque modification du code (Shift-Left Testing).

Cependant, le Runner CI/CD ne sait pas interpréter le texte vert ou rouge affiché par Pester dans la console. Il a besoin d'un rapport structuré (généralement au format XML standardisé **NUnit** ou **JUnit**).

4.1 Exécuter Pester et générer un rapport XML pour la Pipeline

On configure Pester via un objet New-PesterConfiguration pour qu'il génère un fichier de résultats compréhensible par l'interface web de GitLab/GitHub/Azure.

Script : run-tests.ps1 (Exécuté par l'étape de "Test" de la pipeline)

```
$Config = New-PesterConfiguration
```

```
# 1. Spécifier le dossier contenant les tests
```

```
$Config.Run.Path = ".\Tests\"
```

```
# 2. Activer la sortie XML au format NUnit (Standard CI/CD)
```

```
$Config.TestResult.Enabled = $true
```

```
$Config.TestResult.OutputFormat = "NUnitXml"
```

```
$Config.TestResult.OutputPath = ".\test-results.xml"
```

```
# 3. Forcer Pester à faire échouer le script si au moins un test échoue
```

```
$Config.Run.Exit = $true
```

```
# Exécution
```

```
Invoke-Pester -Configuration $Config
```

Si un test Pester échoue, Run.Exit = \$true ordonnera à PowerShell de renvoyer le code exit 1, ce qui bloquera le déploiement. L'orchestrateur lira le fichier test-results.xml et affichera dans son interface web exactement quel bloc It a causé la défaillance.

5. Synthèse : À quoi ressemble le fichier Pipeline (YAML) ?

Pour lier tous ces concepts, voici à quoi ressemble un fichier de configuration CI/CD moderne

(ex: GitLab CI .gitlab-ci.yml ou GitHub Actions) qui utilise nos scripts PowerShell :

Exemple simplifié d'une pipeline CI/CD orientée PowerShell

stages:

- lint
- test
- deploy
- verify

Étape 1 : Analyse Statique du code (J3S4)

AnalyseSyntaxique:

stage: lint

script:

- pwsh -Command "Invoke-ScriptAnalyzer -Path .\Src -EnableExit"

Étape 2 : Tests Unitaires Pester (J5S2)

TestsUnitaires:

stage: test

script:

- pwsh -File .\Scripts\run-tests.ps1

artifacts:

reports:

junit: test-results.xml # Permet d'afficher les résultats dans l'UI

Étape 3 : Déploiement

DeploiementProduction:

stage: deploy

script:

- pwsh -File .\Scripts\deploy.ps1

environment:

name: production

Étape 4 : Audit InSpec / Tests d'intégration (J5S3)

ValidationInfrastructure:

stage: verify

script:

- pwsh -File .\Scripts\run-integration-tests.ps1

6. La Frise du Pipeline DevOps (Infrastructure as Code)

1. **Planification** : Un ticket est créé pour demander un nouveau port ouvert ou un nouveau serveur.
2. **Code** : L'administrateur modifie le script ou le fichier DSC sur son poste.
3. **Commit & Push** : Le code est envoyé sur Git.
4. **Lint (Pipeline)** : PSScriptAnalyzer valide qu'il n'y a pas d'Alias, de mots de passe en dur ou de mauvaises pratiques.
5. **Test (Pipeline)** : Pester exécute les tests unitaires (avec des Mocks) pour s'assurer que la logique ne cassera rien.
6. **Deploy (Pipeline)** : Le script PowerShell (via Invoke-Command ou en poussant un MOF DSC) exécute la modification sur la production.
7. **Verify (Pipeline)** : Des tests d'intégration Pester interrogent le serveur pour s'assurer que le port est *vraiment* ouvert ou que le rôle est *vraiment* installé.